

LAPORAN PRAKTIKUM
STRUKTUR DATA
SINGLE LINKED LIST



OLEH:
DEVINA AMANDA PUTRI
(2411533009)

DOSEN PENGAMPU:
DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2024

A. PENDAHULUAN

Struktur data Single Linked List merupakan salah satu bentuk senarai berantai yang paling sederhana. Setiap elemen dalam Single Linked List, yang disebut sebagai node, terdiri dari dua bagian: data dan pointer ke node berikutnya. Single Linked List bersifat dinamis, artinya alokasi memori dilakukan saat program berjalan, sehingga lebih fleksibel dibandingkan array dalam hal ukuran. Salah satu kelebihan utama linked list adalah kemampuannya untuk menyisipkan dan menghapus elemen dengan efisien, terutama saat operasi tersebut dilakukan di awal atau tengah struktur.

Operasi-operasi dasar biasa dilakukan pada Single Linked List meliputi penambahan node, baik di awal, akhir atau pada posisi tertentu, pencarian elemen berdasarkan nilai, serta penghapusan node dari berbagai posisi. Untuk mengelola operasi tersebut, pemahaman tentang pengaturan pointer antar node menjadi sangat penting. Kesalahan kecil dalam mengatur pointer bisa menyebabkan kehilangan akses ke node tertentu atau bahkan menyebabkan error seperti null pointer exception. Oleh karena itu, pemahaman logika transversal dan manipulasi pointer menjadi inti dalam bekerja dalam struktur data ini

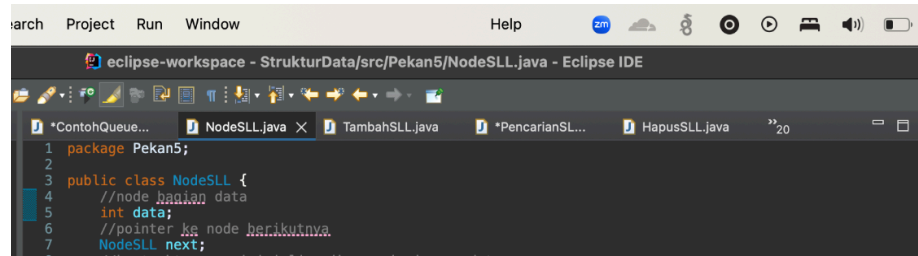
B. TUJUAN

1. Memahami konsep dasar struktur data Single Linked List
2. Mengetahui cara mengimplementasikan node dalam Java

C. LANGKAH-LANGKAH Pengerjaan

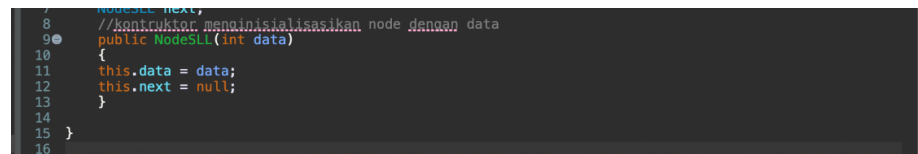
a. Class NodeSLL.java

1. Buat class baru dengan nama NodeSLL dan package Pekan5, deklarasikan class NodeSLL yang didalamnya terdapat 2 atribut: data untuk menyimpan nilai yang dimiliki node dan next yang menunjukkan ke node berikutnya dalam linked list



```
1 package Pekan5;
2
3 public class NodeSLL {
4     //node akan data
5     int data;
6     //pointer ke node berikutnya
7     NodeSLL next;
8 }
```

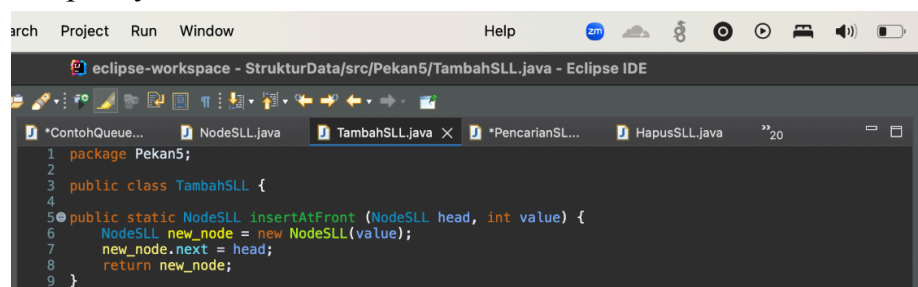
2. public NodeSLL(int data) merupakan konstruktor yang digunakan untuk membuat node baru dengan nilai data, saat node dibuat, next diset ke null karena belum terhubung ke node lain.



```
7 NodeSLL next;
8 //konstruktor menginisialisasi node dengan data
9 public NodeSLL(int data)
10 {
11     this.data = data;
12     this.next = null;
13 }
14
15 }
16 }
```

b. Class TambahSLL.java

1. Buat class baru dengan nama TambahSLL pada package Pekan5. Tambahkan fungsi untuk menambahkan node baru di depan linked list. Node baru akan menunjuk ke node lama yang tadi ada didepannya.



```
1 package Pekan5;
2
3 public class TambahSLL {
4
5     public static NodeSLL insertAtFront (NodeSLL head, int value) {
6         NodeSLL new_node = new NodeSLL(value);
7         new_node.next = head;
8         return new_node;
9     }
10 }
```

2. Pada fungsi ini menambahkan node di akhir linked list, jika linked list kosong langsung membuat node baru, jika tidak

looping akan sampai node terakhir lalu bersambung ke node baru.

```
9 }
10 //fungsi menambahkan node di akhir SLL
11 public static NodeSLL insertAtEnd (NodeSLL head, int value) {
12     //buat node dengan nilai
13     NodeSLL newNode = new NodeSLL(value);
14     //link list kosong
15     if (head == null) {
16         return newNode;
17     }
18     NodeSLL last = head;
19     while (last.next != null) {
20         last = last.next;
21     }
22     last.next = newNode;
23     return head;
24 }
25 static NodeSLL GetNode(int data) {
```

3. Menambahkan fungsi bantu untuk membuat node baru, agar kode di fungsi lain lebih ringkas

```
24 }
25 static NodeSLL GetNode(int data) {
26     return new NodeSLL(data);
27 }
28 }
```

4. Membuat fungsi untuk menambahkan node di posisi tertentu lalu di cek apakah posisi nya valid. Jika posisi yang diminta adalah yang paling depan, langsung insert seperti biasa, namun jika bukan posisi 1 akan dilanjutkan dengan perulangan untuk cari posisi yang sesuai. Kalau berhasil node baru akan disisipkan ditengah, kalau akan tidak muncul error

```
29 static NodeSLL insertPos(NodeSLL headNode, int position, int value) {
30     NodeSLL head = headNode;
31     if (position < 1)
32         System.out.print("Invalid position");
33
34     if (position == 1) {
35         NodeSLL new_node = new NodeSLL(value);
36         new_node.next = head;
37         return new_node;
38     } else {
39         while (position-- != 0) {
40             if (position == 1) {
41                 NodeSLL newNode = GetNode(value);
42                 newNode.next = headNode.next;
43                 headNode.next = newNode;
44                 break;
45             }
46             headNode = headNode.next;
47         }
48         if (position != 1)
49             System.out.print("Posisi di luar jangkauan");
50     }
51     return head;
52 }
53 }
```

5. Fungsi printList untuk menelusuri dan mencetak isi linkedlist, buat variabel curr dan di set nilainya sama dengan head, curr akan digunakan untuk menelusuri linked list dari node pertama sampai node terakhir. Dengan perulangan while, selama node berikutnya dari curr tidak null berarti masih ada node yang bisa dicetak, loop ini akan berhenti saat curr berada di node terakhir lalu cetak nilai yang di simpan di node saat ini, diikuti tanda (->) untuk menunjuk ke node berikutnya. Pindahkan curr ke node selanjutnya agar bisa di cek dan dicetak di iterasi berikutnya.

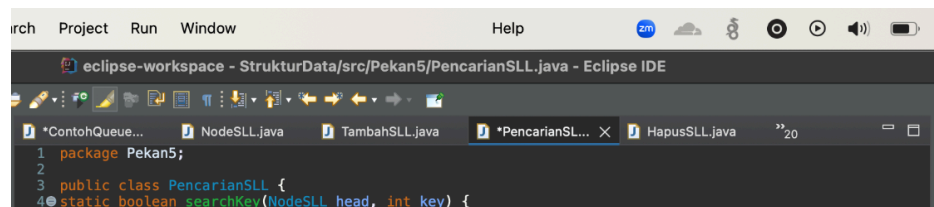
```
54 public static void printList(NodeSLL head) {
55     NodeSLL curr = head;
56     while (curr.next != null) {
57         System.out.print(curr.data+"-> ");
58         curr = curr.next;
59     }
60     if (curr.next==null) {
61         System.out.println(curr.data);}
62     System.out.println();
63 }
64 }
```

6. Pada fungsi main, membuat linked list awal, lalu cetak isi awal linkedlist. Tambah node 1 di depan dan node 7 di akhir, tambahkan node 4 di posisi ke-4 lalu menampilkan isi linked list

```
65
66 public static void main(String[] args) {
67     NodeSLL head= new NodeSLL(2);
68     head.next = new NodeSLL(3);
69     head.next.next= new NodeSLL(5);
70     head.next.next.next= new NodeSLL(6);
71
72     System.out.print("Senarai berantai awal: ");
73     printList(head);
74
75     // tambahkan node baru di depan
76     System.out.print("tambah 1 simpul di depan: ");
77     int data = 1;
78     head = insertAtFront(head, data);
79     // cetak update list
80     printList(head);
81
82     // tambahkan node baru di belakang
83     System.out.print("tambah 1 simpul di belakang: ");
84     int data2 = 7;
85     head = insertAtEnd(head, data2);
86     // cetak update list
87     printList(head);
88
89     System.out.print("tambah 1 simpul ke data 4: ");
90     int data3 = 4;
91     int pos = 4;
92     head = insertPos(head, pos, data3);
93     // cetak update list
94     printList(head);
95 }
96
97 }
98
99
```

c. Class PencarianSLL.java

1. Buat class baru dengan nama PencarianSLL pada package yang sama, buat fungsi untuk mencari apakah suatu nilai ada dalam linked list dengan tipe pengembalian boolean, artinya akan mengembalikan true kalau ketemu dan false kalau tidak.



```
1 package Pekan5;
2
3 public class PencarianSLL {
4     static boolean searchKey(NodeSLL head, int key) {
```

2. Inisialisasikan pointer curr untuk mulai dari node paling depan, lakukan pencarian dengan loop while selama curr belum mencapai akhir list. Jika data dalam node saat ini sama dengan nilai yang dicari, langsung kembalikan true. Lalu pindah ke node berikutnya, kembalikan false apabila loop selesai tanpa menemukan data

```
5     NodeSLL curr = head;
6     while (curr != null) {
7         if (curr.data == key)
8             return true;
9         curr = curr.next; }
10    return false;
11 }
```

3. Membuat fungsi transversal untuk menelusuri dan mencetak semua isi dari linked list, mulai dari node paling depan. Dengan perulangan while, selama masih ada node maka tampilkan isinya

lalu pindah ke node berikutnya.

```
11 }
12 public static void transversal(NodeSLL head) {
13     NodeSLL curr = head;
14     while (curr != null) {
15         System.out.print(" " + curr.data);
16         curr = curr.next;
17     }
18     System.out.println();
19 }
```

4. Pada fungsi main, buat 5 node dan hubungkan menjadi satu linked list dengan head.next dan seterusnya. Tampilkan isi list dengan memanggil fungsi transversal.

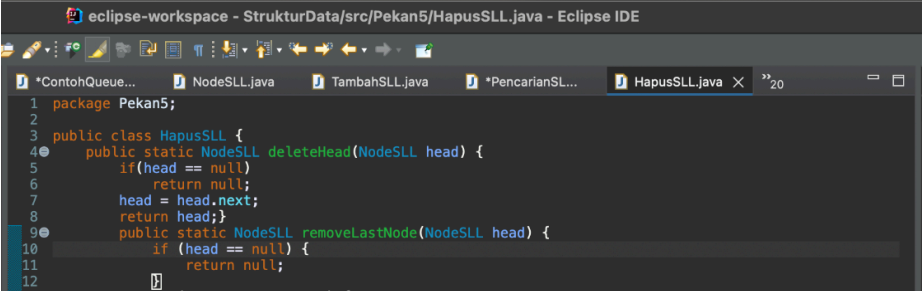
int key = 30;

System.out.print("cari data " + key + " = ");, untuk mencari angka 30 ada didalam list, dan memanggil fungsi searchKey. Kalau hasilnya true maka cetak "ketemu" kalau tidak cetak "tidak ada"

```
20 public static void main(String[] args) {
21     NodeSLL head = new NodeSLL(14);
22     head.next = new NodeSLL(21);
23     head.next.next = new NodeSLL(13);
24     head.next.next.next = new NodeSLL(30);
25     head.next.next.next.next = new NodeSLL(10);
26
27     System.out.print("Penelusuran SLL : ");
28     transversal(head);
29
30     // data yang akan dicari
31     int key = 30;
32     System.out.print("cari data " + key + " = ");
33     if (searchKey(head, key))
34         System.out.println("ketemu");
35     else
36         System.out.println("tidak ada");
37 }
38 }
39 }
40 }
```

d. Class HapusSLL.java

1. Buat class baru dengan nama HapusSLL.java pada package yang sama. Fungsi pertama yaitu fungsi untuk menghapus node paling depan, kalau linked list kosong, maka langsung return null. Jika tidak kosong, ubah head menjadi node berikutnya (head.next) dan return head baru.



```
1 package Pekan5;
2
3 public class HapusSLL {
4     public static NodeSLL deleteHead(NodeSLL head) {
5         if (head == null)
6             return null;
7         head = head.next;
8         return head;
9     }
10    public static NodeSLL removeLastNode(NodeSLL head) {
11        if (head == null) {
12            return null;
13        }
14        if (head.next == null) {
15            return null;
16        }
17    }
```

2. Membuat fungsi removeLastNode, Jika list kosong, kembalikan null, jika list hanya berisi satu node, setelah dihapus maka akan jadi kosong. Untuk mencari node kedua dari belakang dengan menelusuri sampai next.next == null, lalu putuskan sambungan ke

node terakhir dan kembalikan list yang sudah di potong.

```
9      return head;
10     public static NodeSLL removeLastNode(NodeSLL head) {
11         if (head == null) {
12             return null;
13         }
14         if (head.next == null) {
15             return null;
16         }
17         NodeSLL secondLast = head;
18         while (secondLast.next.next != null) {
19             secondLast = secondLast.next;
20         }
21         secondLast.next = null;
22         return head;
23     }
24 }
```

3. Pada fungsi deleteNode, fungsi ini untuk menghapus node di posisi tertentu, cek dahulu apakah list kosong. Jika posisi yang ingin dihapus adalah posisi pertama, hapus head dengan menggeser ke node berikutnya. Lalu cari posisi node yang ingin dihapus, temp akan menunjuk ke node target, prev ke node sebelumnya. Kalau temp tidak null yang berarti valid posisinya, lewati node temp dengan menyambungkan prev.next ke temp.next

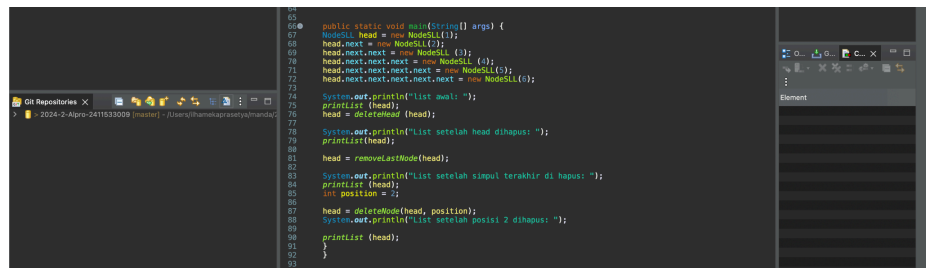
```
25     public static NodeSLL deleteNode(NodeSLL head, int position) {
26         NodeSLL temp = head;
27         NodeSLL prev = null;
28
29         if (temp == null)
30             return head;
31
32         if (position == 1) {
33             head = temp.next;
34             return head;
35         }
36
37         for (int i = 1; temp != null && i < position; i++) {
38             prev = temp;
39             temp = temp.next;
40         }
41
42         if (temp != null) {
43             prev.next = temp.next;
44         } else {
45             System.out.println("Data tidak ada");
46         }
47
48         return head;
49     }
50 }
```

4. Fungsi printList untuk menelusuri dan mencetak isi linkedlist, buat variabel curr dan di set nilainya sama dengan head, curr akan digunakan untuk menelusuri linked list dari node pertama sampai node terakhir. Dengan perulangan while, selama node berikutnya dari curr tidak null berarti masih ada node yang bisa dicetak, loop ini akan berhenti saat curr berada di node terakhir lalu cetak nilai yang di simpan di node saat ini, diikuti tanda (-->) untuk menunjuk ke node berikutnya. Pindahkan curr ke node selanjutnya agar bisa di cek dan dicetak di iterasi berikutnya.

```
51     public static void printList(NodeSLL head) {
52         NodeSLL curr = head;
53         while (curr != null) {
54             System.out.print(curr.data + "-->");
55             curr = curr.next;
56         }
57         if (curr.next == null) {
58             System.out.print(curr.data);
59         }
60         System.out.println();
61     }
62 }
```

5. Pada fungsi utama program, buat linked list dengan head.next dan seterusnya. Cetak isi awal list, hapus node pertama lalu cetak lagi. Hapus node terakhir lalu cetak dan hapus node pada posisi

ke-2, lalu cetak lagi.



```
60
61
62 public static void main(String[] args) {
63     NodeSLL head = new NodeSLL(1);
64     head.next = new NodeSLL(2);
65     head.next.next = new NodeSLL(3);
66     head.next.next.next = new NodeSLL(4);
67     head.next.next.next.next = new NodeSLL(5);
68     head.next.next.next.next.next = new NodeSLL(6);
69
70     System.out.println("List awal: ");
71     printList(head);
72     head = deleteHead(head);
73     System.out.println("List setelah head dihapus: ");
74     printList(head);
75     head = removeLastNode(head);
76     System.out.println("List setelah simpul terakhir di hapus: ");
77     printList(head);
78     int position = 1;
79     head = deleteNode(head, position);
80     System.out.println("List setelah posisi 2 dihapus: ");
81     printList(head);
82 }
83 }
```

D. KESIMPULAN

Pada praktikum pekan 5 ini, telah dipelajari konsep dasar Single Linked List dengan berfokus pada implementasi node tunggal dan operasi-operasi dasar seperti penambahan, pencarian serta penghapusan node, memerlukan pemahaman yang cermat terhadap hubungan antar node melalui pointer next. Kesalahan kecil seperti tidak memperbarui pointer bisa menyebabkan referensi hilang atau linked list terputus.